

LoRa for Long-Range and Low-Cost IoT Applications

Mayur Rajaram Parate¹ and Ankit A. Bhurane²

¹Indian Institute of Information Technology, Nagpur

²Visvesvaraya National Institute of Technology, Nagpur

Abstract

LoRa (Long Range) facilitates low-power, long-range, and high-immune communication utilizing sub-gigahertz and license-free radio frequency bands. The data rates supported are between 0.3 kbit/s to 27 kbit/s which is ideal for IoT applications. In this chapter, we begin by briefing about the basic concept of LoRa and its topologies including LoRaWAN. We explore the functionalities of heltec ESP32-LoRa development platform and cover several demonstrations to get started with the LoRa protocol and its applications. Heltec ESP32-LoRa is an attractive solution for building long and short range IoT networks as it is equipped with Semtech's SX1278 LoRa RF Module and ESP32 which is a powerful, low-cost microcontroller popular for IoT applications. We cover data acquisition and processing using widely used sensors like temperature, proximity, gas, SPo2, Heart-rate, etc. are demonstrated. Some examples for on-board OLED displays and a standard communication model are also demonstrated with minimal overheads. Finally, applications related to industry and smart agriculture which demand long range communication and power efficiency are demonstrated.

Keywords: SX1278 LoRa RF Module, ESP32, Long range IoT, LoRaWAN, Long range RF Communication, Low Power Wide Area Networks (LPWAN)

1 Introduction to LoRa and Scope of the Chapter

Long range (LoRa) is a patented frequency modulation technology developed by Semtech Corporation that provides communication over a wide area networks finding applications majorly in internet of things (IoT).

Some salient features of LoRa are,

1. Long range: LoRa can facilitate communication range up to as long as 15 Km and provides penetrated coverage indoors without GPS.
2. Low power: LoRa is designed for low-power applications and ensures extended battery life for IoT applications. The node power is adapted as per the distance to the gateway (GW) and spread.
3. High capacity: LoRa is capable to handle multiple thousands of message transactions in a day thus making it suitable for public wide networks.
4. Open-source software: The software being open source keeps the software overhead to minimal.
5. Security: LoRa is protected by AES-128 encryption technique.
6. Robust communication: LoRa signals are immune to in and out of the band interference, time and frequency shifts.

LoRa is based on chirp spread spectrum (CSS) technique where the data bits are bandwidth spread by multiplying the data bits by a spreading code (also called as chip sequence) [9]. The spectrum spreading of the signal is controlled by various spreading factors (SF) which also determines the range of communication. The signals with different SF if transmitted in same time slot do not overlap with each other.

2 LoRa Standard, Topologies (LoRaWAN), Frequency Bands (Asia)

A typical single and multi-node topologies of a LoRaWAN network is shown in Figure 1 and 20. The general operation of a LoRaWAN network is as follows.

1. End device: An end device/ node can be any wireless device capable of sensing or actuating a signal. These are usually battery-operated devices. The data from the end device (say sensor) is up-linked to the hearing GW. There can be multiple GW deployed to ensure high reception rate.
2. GW: GW are responsible for reception and forwarding of the messages sent by the end nodes. The messages are further sent to central network server. Entire transaction of messages through GW happen at the lowest (physical) layer.
3. Network servers (NS): NS handles the end-to-end networking, relaying to application server, securing, authenticity, acknowledgement and transport of the data.
4. Application servers (AS): The user end management, application packet downloads, delivery and connectivity are all managed by the respective AS.

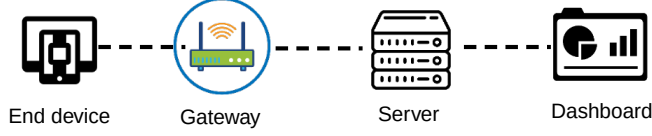


Figure 1: A typical LoRaWAN network

Region	India	Asia	Europe	United States	Australia	South Korea
Frequency band (MHz)	865-867	923	863-870	902-928	915-928	920-923

Table 1: Region-wise LoRa bands

In addition to network and application servers, the overall join-request-accept of uploading and downloading, storage and verification of encryption keys is managed by an additional *join server* (JS).

3 Basic steps of end-to-end operation of a LoRa network

Step 1: An end node transmits a *send request* signal to JS.

Step 2: Post authentication, the JS accepts and acknowledge request to the respective end node.

Step 3: Both end node and join server generates session keys.

Step 4: The connection between the end node, network server and application server is established using respective session keys. The sessions being secured with 128-bit AES ensures data confidentiality with NS and GW.

Step 5: Data is exchanged between end node and intended application.

LoRa operates at different frequencies in different regions. For some regions the bands are shown in Table 1. A complete list of LoRa bands can be obtained in [2].

4 Overview of ESP32 based LoRa platform

Internet-of-things (IoT) best defined by the network connecting the physical objects ("things") that employ sensors, communication capabilities, and softwares to communicate with other devices over the Internet and exchange information. IoT have evolved as a amalgamation of multiple technologies that need

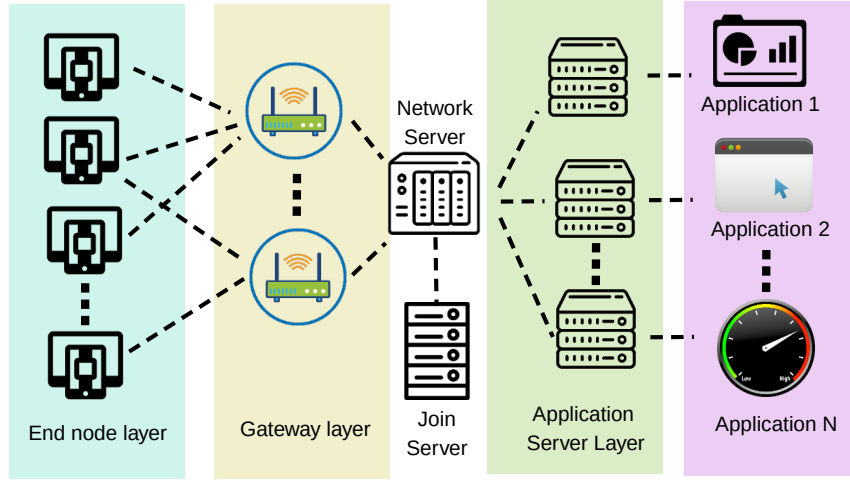


Figure 2: A typical LoRaWAN network with multiple end nodes, gateways and applications

powerfull computing capabilities, good communication range and low power requirement. Here, we introduce ESP32 as a computing element and LoRa as a communication technology in a IoT device.

4.1 Introduction to ESP32

ESP32 is a series of low-power, low-cost system-on-chip (SOC) microcontrollers Espressif Systems, a Shanghai-based company. ESP32 is manufactured using 40nm technology by TSMC. Being SOC, it is integrated with on chip dual-mode Bluetooth and full stack Wifi. The ESP32 is enabled with either a Tensilica Xtensa LX6 microprocessor in dual-core or single-core variations or Xtensa LX7 dual-core microprocessor or single-core RISC-V microprocessor. It also includes power amplifier, low-noise receive amplifier, built-in antenna switches, RF balun, filters, and power-management modules [5].

ESP32 can perform as a complete standalone system or can act as a slave device for a host Microcontroller to reduce communication stack overhead on the main application processor. Further, ESP32 can also be used to provide WiFi and Bluetooth capabilities through its SPI / SDIO or I2C / UART interfaces.

The ESP32 are designed to sustain industrial environment and enabled with advanced calibration circuitries. It is capable of dynamically reducing imperfections in the external circuit and adapt to changes in external conditions. Moreover, it can sustain the temperature in industrial scenario i.e. from -40°C to $+125^{\circ}\text{C}$.

ESP32 achieves low power consumption employing its various power modes and dynamic power scaling. It also has a ultra-low power core which is ideal for mobile devices, wearable electronics and IoT applications.

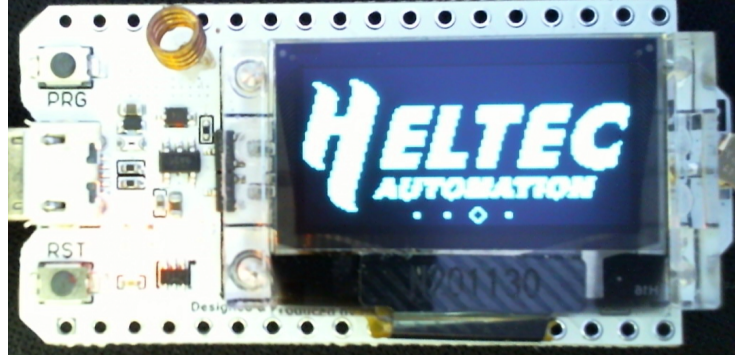


Figure 3: Heltec ESP32 LoRa Module

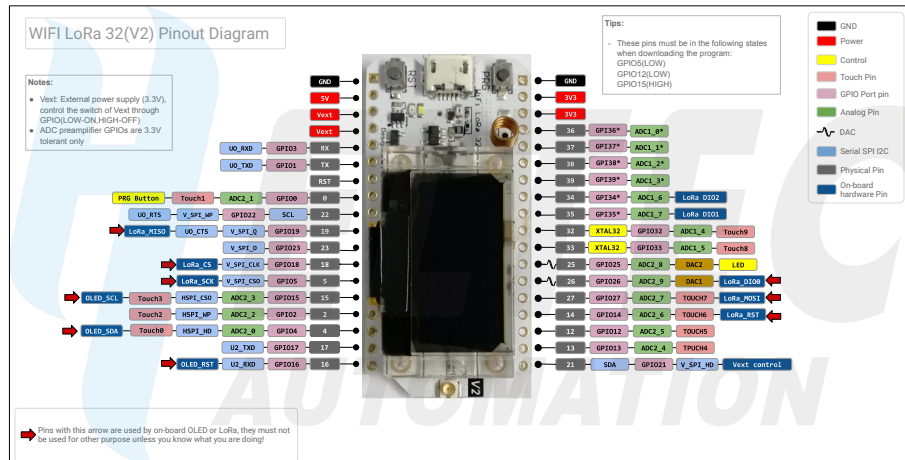


Figure 4: Pin diagram for Heltec ESP32 LoRa Module [6]

4.2 Introduction to Heltec LoRa Module

LoRa is low power, long range, wireless platform which became ideal for Internet of Things (IoT). LoRa devices and networks such as the LoRaWAN® stands in a Low Power, Wide Area Network (LPWAN) category that enable smart IoT applications to communicate over a long distance using very less power. An ESP32 based LoRa module by Heltech Automations is a complete development board with GPIOs and have surface mount 0.96 Inch Blue OLED Display [6] as shown in Fig.3. Due to its on-board display and ease of programming using Arduino, it is very simple and time efficient option for prototyping, program validation and product development.

The heart of ESP32 is Tensilica LX6 dual-core processor operating at a clock speed of 240MHz. The chip has built-in 520 KB SRAM, full stack (802.11 b/g/N) HT40 WiFi and Lightweight IP (LWIP) Bluetooth enabled with Blue-

tooth low power technology (BLE). Further, the chip is powerfull enough to execute 600 million instructions per second.

To add extra functionality in the ESP32, Heltec added SX1278 chip, which is a LoRa remote modem and support -148dBm high sensitivity at +20 dBm output power. The main objective behind this integration is to achieve highly reliable long distance transmission with all the features supported by ESP32. The communication range achieved using Heltec ESP32 LoRa development board measured in open area is 2.6Km.

The ease of programming is achieved by integrating CP2102 USB to serial chip on the board. It support Arduino integrated development environment and can be used with all other available libraries for sensors and interfaces. The pin diagram for the ESP32 based LoRa module by Heltech Automations is shown in Fig. 4.

5 Get-Started “Hello/ Welcome!” program

In order to make programming and development user friendly, we illustrate development of IoT nodes using “arduino” Integrated Development Environment (IDE). We are using ‘Heltec esp32-LoRa Development Board’ to demonstrate integration of ESP32 and LoRa with Arduino IDE. To do this, we need to install arduino IDE and required libraries and dependencies.

5.1 Arduino IDE installation

Arduino is an open source IDE for programming various microcontrollers of Atmel, STM32, ESP32, etc. [3]. This section illustrate the process of installing arduino IDE and required dependencies for ESP32 with LoRa.

- Download Arduino IDE as shown in Fig.5 from “<https://www.arduino.cc/>”

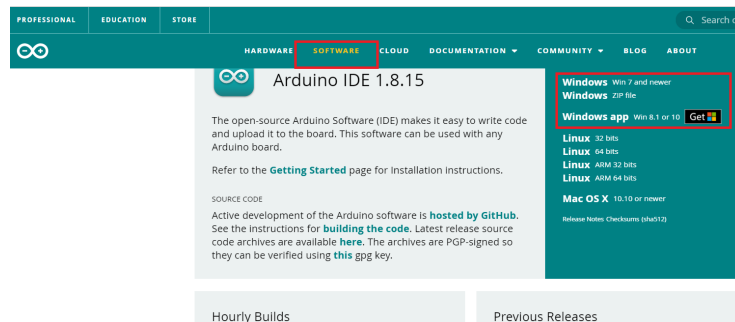


Figure 5: Arduino Website

- Install the IDE suitable for your operating system.

- Open the IDE, go to 'File' and then to 'Preferences' as shown in Fig.6.

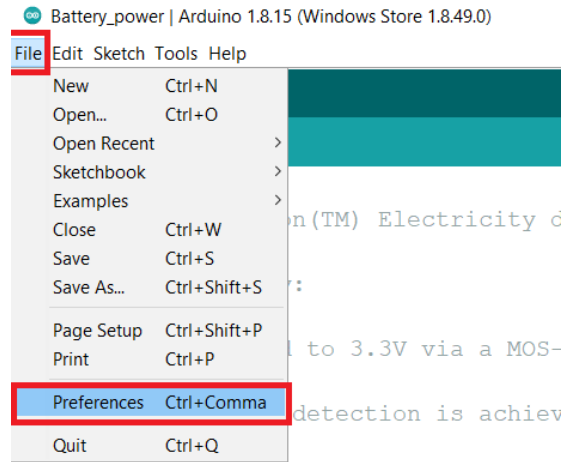


Figure 6: Go to 'File' and then to 'Preferences'

- In Preferences window, go to 'Additional Boards Manager URLs' and paste the following URL and click 'OK'.
https://github.com/Heltec-Aaron-Lee/WiFi_Kit_series/releases/download/0.0.5/package_heltec_esp32_index.json
 Adding this URL allows Arduino IDE to capture libraries and boards from the repository.
- Now, go to 'Tools' and select 'Boards Manager' option as shown in Fig.7.

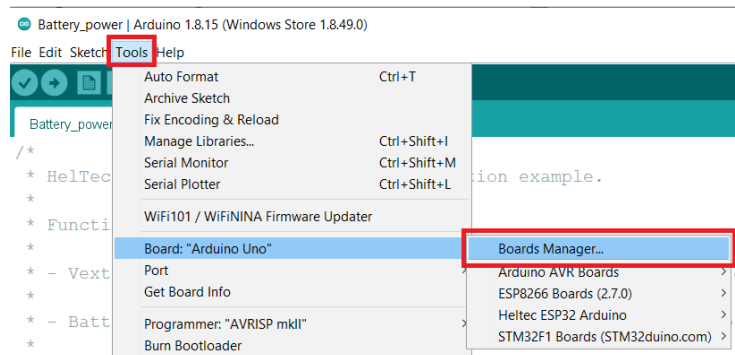


Figure 7: Go to 'tools' and then 'Board Manager'

- In Boards Manager window, search for ‘heltec ESP32’ and install the latest version of boards by “Heltec ESP32 Series Dev-Boards” as shown in Fig.8.

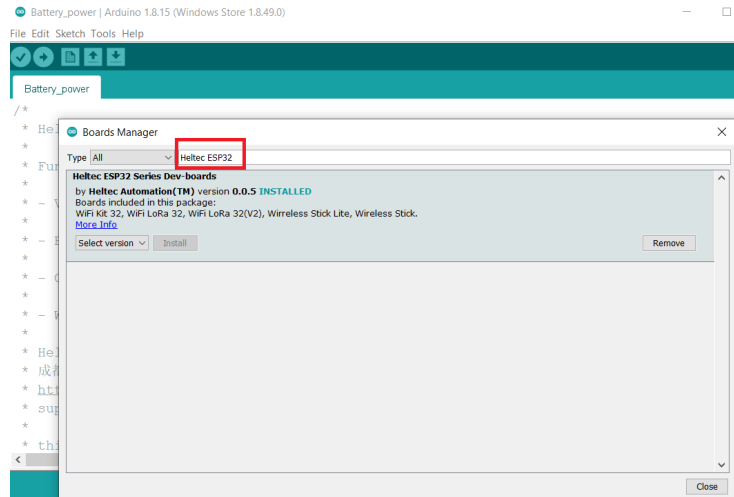


Figure 8: In Board Manager window search for ‘Heltec ESP32’ and ‘Install’

- Now, again in ‘Tools’, go to ‘Manage Libraries’ option to add corresponding libraries from the repository.
- In ‘Library Manager’ search for ‘heltec ESP32’ and install the latest version of libraries by “Heltec ESP32 Dev-Boards” as shown in Fig.10. At this point all the dependencies and libraries are installed in Arduino IDE for ESP32-LoRa Development Board.
- After completing all installations, we select the appropriate board for programming.
Go to Tools -> Board -> Heltec ESP32 Arduino -> WiFi LoRa 32(v2). Here, we are using ‘WiFi LoRa 32(v2)’, if you have any other version then choose accordingly.
- Arduino IDE have facility to program a supported device by serial communication through USB using CP210X USB to UART bridge. It creates a virtual serial COM port in a computer. The COM port corresponding to the connected device can be verified at ‘Device Manager’ in a Windows machine as shown in a Fig.12.

To program a connected device, select the same ‘COM Port’ in arduino IDE at Tools -> Port.

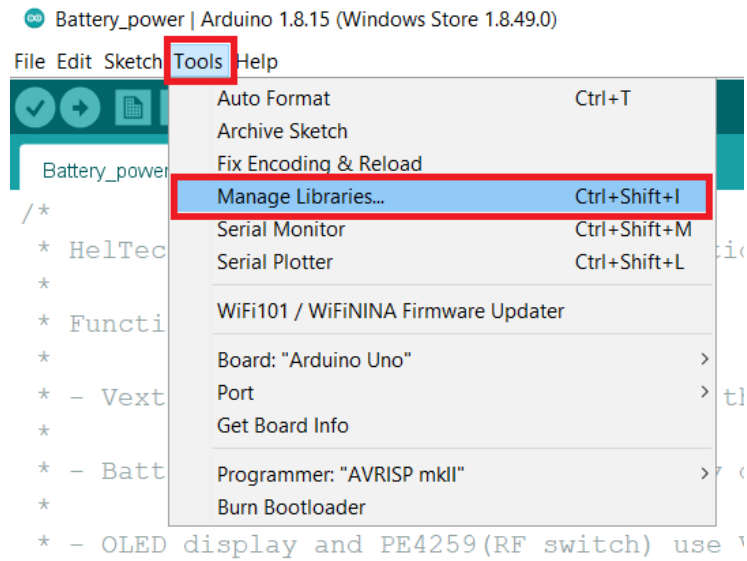


Figure 9: In 'Tools' go to 'Manage Libraries'

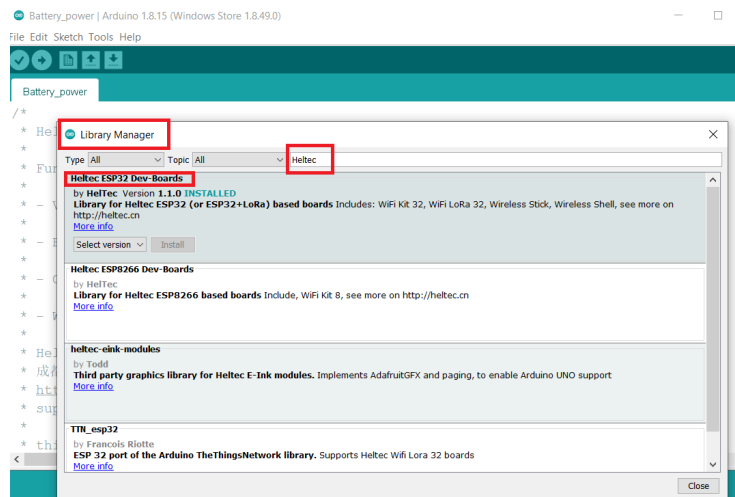


Figure 10: In 'Library Manager' window search for 'Heltec ESP32' and click 'Install'

5.2 Testing The Development Board

With this setting, the development board can be tested for its various functionalities [7].

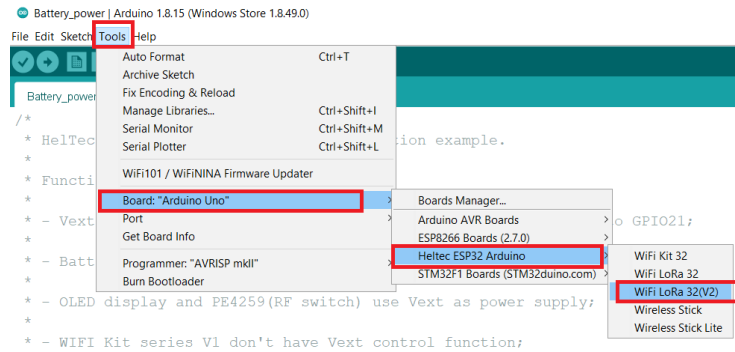


Figure 11: Select the appropriate Board

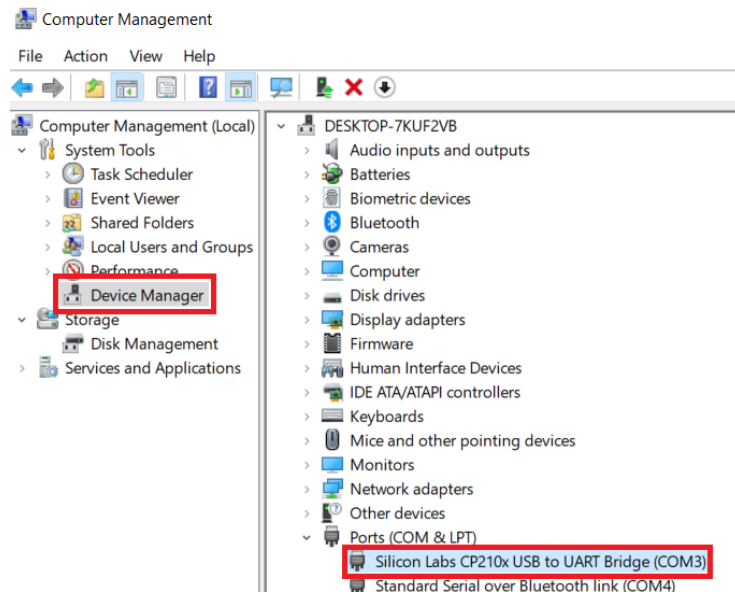


Figure 12: Verify COM port at Device Manager in Windows Machine

5.2.1 Testing the basic functionalities

The board supports multiple functionalities and needed to be tested before using in a complex development. The factory test program test the development board for the following

- Basic WiFi function testing
We test the wifi functionalities of the development board by setting wifi in a station mode. If the board is already used and have SSID and Password saved in it, then it connects automatically otherwise use provided SSID and Password for new connection.

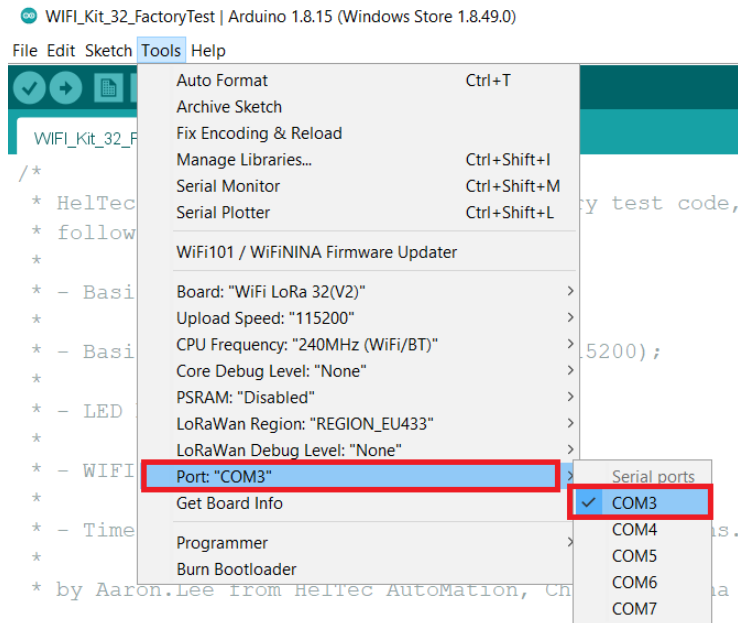


Figure 13: Select respective COM Port for connected device

Listing.1 presents the code for wifi-initialization and testing its functionalities, `#include "heltec.h"` and `#include "WiFi.h"` include the header files 'heltec.h' and 'WiFi.h'. These files are developed by Heltec Automation and provide definitions for the functions we will be using in the `setup()`. `WiFi.mode(WIFI_STA)` on line 8, sets the wifi in station mode so that the development board can connects to provided SSID and Password in `WiFi.begin("SSID", "Password")`.

```

1      #include "heltec.h"
2      #include "WiFi.h"
3      void setup()
4      {
5          // Setup a fresh WiFi station
6          WiFi.disconnect(true);
7          delay(2000);
8          WiFi.mode(WIFI_STA); //select station mode
9          WiFi.setAutoConnect(true);
10         WiFi.begin("SSID","Password");
11         delay(200);
12     }
13
14     void loop()
15     {
16         // your main code will come here.
17     }
18

```

Listing 1: Code for testing WiFi

- Basic OLED function test

We initialize OLED display and print “Hello!” on the screen. It is mandatory to initialize the Heltec Object as follows.

```
Heltec.begin(Enable-Display, Enable-LoRa, Enable-Serial-Communication,
Enable-Antenna-Power-Boost , BAND);
```

The argument “true” and “false” provided to a function `Heltec.begin()` either enable or disable the corresponding functionality in the heltec development board. First four argument are Boolean whereas the last argument is the LoRa band.

The lines 8 to 10 indicate, the initialization of OLED display, flipping the screen vertically for horizontal view, setting font to ‘ArialMT_Plain’ and text size to 10. Line 11, display the string “It’s an OLED Display” starting from line position 0 and character position 0. The required simple setup is shown below in Listing.2.

```

1      #include "heltec.h"
2      #include "images.h"
3      #define BAND    865E6  // Set LoRa band (here Indian)
4      void setup()
5      {
6          Heltec.begin(true, true, true, true, BAND);
7          // Here we setup the settings for OLED Display
8          Heltec.display->init();
9          Heltec.display->flipScreenVertically();
10         Heltec.display->setFont(ArialMT_Plain_10);
11         Heltec.display->drawString(0, 0, "It's an OLED
Display");
12         Heltec.display->display();
13         delay(1000);
14     }
15
16     void loop()
17     {
18         // your main code will come here.
19     }
20

```

Listing 2: Code for testing OLED display

- Basic LoRa function Test

Basic functionalities of the LoRa can be tested using the following code. Initially, the frequency band allowed as per the region needs to be selected for communication over the LoRa. “Heltec.begin()” function initializes LoRa and other features in the development board as explained in ‘Basic OLED function test’. The data packet of the LoRa needs initialization and termination as provided by “LoRa.beginPacket()” and “LoRa.endPacket()” and data to be send is placed in-between these function using “LoRa.print(data)”. The example code is shown in Listing.3. In this, the data we are sending over the LoRa-Network is “hello”.

```

1      # include "heltec.h"
2      #define BAND    865E6    // Set LoRa band (here Indian)
3      void setup()
4      {
5          Heltec.begin(true, true, true, true, BAND);
6          delay(2000);
7      }
8      void loop()
9      {
10         // send packet
11         LoRa.beginPacket(); // starts the data packet
12         formation
13         LoRa.setTxPower(14,RF_PACONFIG_PASELECT_PABOOST); //
14         sets the transmission power
15         LoRa.print("hello "); // incapsulate the data into
16         the LoRa packet
17         LoRa.endPacket(); // ends and transmit the data
18         packet
19     }

```

Listing 3: Code for testing LoRa functionality

- Basic LoRa transmitter and Receiver

We studied the simple LoRa configuration in previous topic, whereas, how to set LoRa module into transmitter and receiver is explored here. The transmitter and receiver can be obtained by configuring one LoRa kit in transmitter mode using `LoRa.setTxPower(txPower,RFOUT_pin)` as in line 13 in Listing .4. and other LoRa kit in the receiving mode using `LoRa.read()` commands as in line 13 in Listing.5. During transmission, the `RFOUT_pin` could be `RF0` to attain a maximum output power of 14 dBm or `PABOOST` to achieve maximum of 20 dBm. A simple working example for transmitter and receiver is given. In transmitter, we are sending data as "Sent Packet:" followed by the packet number.

At receiver, we parse the received packet for data at line 9 and print the same data on serial port at line 13. The listings and the output for the same is shown in Fig. 14.

```

1  #include "heltec.h"
2  #define BAND 868E6 // Set LoRa band (here Indian)
3  int Packet_Number = 0;
4  void setup() {
5      // Enable display, LoRa, serial communication with
        antenna power boost using Indian band
6      Heltec.begin(true, true, true, true, BAND);
7  }
8  void loop() {
9      Serial.print("Sending packet: "); // print details on
        serial terminal of arduino
10     Serial.println(Packet_Number);
11     //Send packet with maximum power of 20 dBm
12     LoRa.beginPacket(); // start packet
13     LoRa.setTxPower(20,RF_PACONFIG_PASELECT_PABOOST);
14     LoRa.print("Sent Packet: "); // include data into packet
15     LoRa.print(Packet_Number); // include data into packet
16     LoRa.endPacket(); // ends the packet
17     Packet_Number++;
18 }
19

```

Listing 4: Code to setup a LoRa transmitter

```

1  #include "heltec.h"
2  #define BAND 868E6 // Set LoRa band (here Indian)
3  void setup() {
4      // Enable display, LoRa, serial communication with
        antenna power boost using Indian band
5      Heltec.begin(true, true, true, true, BAND);
6  }
7  void loop() {
8      // Receive packet
9      int pktSz = LoRa.parsePacket(); // parse the received
        packet and calculate its packet size
10     if (pktSz) {
11         // If packet print "Received" and packet data
12         Serial.print("Received ");
13         // read packet
14         while (LoRa.available()) {
15             Serial.print((char)LoRa.read()); // print the
                received data from LoRa on to the serial terminal of arduino.
16         }
17         Serial.print(" \n");
18     }
19 }

```

Listing 5: Code to setup a LoRa receiver

- Basic serial port test(in baud rate 115200); Support for the serial communication can be tested simply at serial terminal of an Arduino. The serial communication in the development board can be enabled using the `Heltec.begin()` function by making corresponding argument as 'true' as

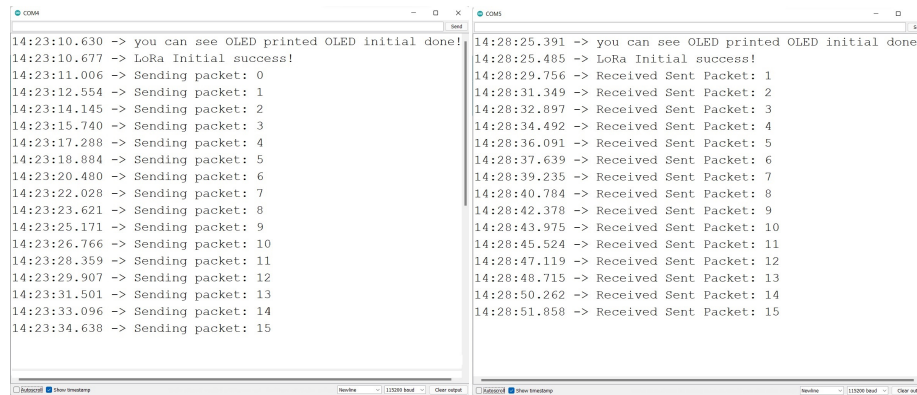


Figure 14: Serial monitor output of a simple packet transmitter-receiver

in line 5 of Listing.6. The data to be printed can be sent to serial port using `Serial.printf()` command as in line 7.

```

1      #include "heltec.h"
2      #define BAND    865E6  // Set LoRa band (here Indian)
3      void setup()
4      {
5          Heltec.begin(true, true, true, true, BAND); //
        initialize the serial communication
6          delay(1000);
7          Serial.printf("Hello!"); // Print data serially.
8      }
9      void loop()
10     {
11     }
12

```

Listing 6: Code for Serial communication initialization

- Other Arduino basic functions such as timers, interrupts, etc. can be tested as per the requirement.

The complete and comprehensive code for the testing development board for its communication using WiFi is available in Listing.5. The output for the same on the inbuilt LCD screen is shown in Fig. 15.

The provided listing is the combination of listings 1 to 6, in which `void WIFISetUp(void)` function setup the wifi and display its connection status. Specifically, code from line 8 to 13 initialize the wifi. The connection status during the wifi connection is printed on the OLED display at lines 16 to 39. The function `void WIFIScan(void)` at lines 41 to 83 scans the available wifi in the area and display those wifi SSIDs on OLED display. Then in `void setup()`, we begin the heltec development board and call the above declared functions.

```

1
2 #include "heltec.h"
3 #include "WiFi.h"
4
5 void WIFISetUp(void)
6 {
7     // Setup a fresh WiFi station
8     WiFi.disconnect(true);
9     delay(2000);
10    WiFi.mode(WIFI_STA); // set wifi in station mode
11    WiFi.setAutoConnect(true);
12    WiFi.begin("SSID","Password");
13    delay(200);
14
15    byte count = 0;
16    while(WiFi.status() != WL_CONNECTED && count < 10)
17    {
18        count++; // wait for 10 seconds for connection
19        delay(1000);
20        Heltec.display -> drawString(0, 0, "Connection in Progress..."); //
21        // display connection status on OLED display
22        Heltec.display -> display();
23    }
24    Heltec.display -> clear();
25    if(WiFi.status() == WL_CONNECTED) // if true then connection
26    // successful
27    {
28        Heltec.display -> drawString(0, 0, "Connected, congratulations!");
29        Heltec.display -> display();
30    }
31    else
32    {
33        Heltec.display -> clear();
34        Heltec.display -> drawString(0, 0, "Connection ");
35        Heltec.display -> display();
36    }
37    // while(1);
38    Heltec.display -> drawString(0, 15, "WIFI Setup done");
39    Heltec.display -> display();
40    delay(500);
41 }
42
43 void WIFIScan(void) // scans for the available wifi connections in the
44 // area
45 {
46     Heltec.display -> drawString(0, 15, "Started Scanning");
47     Heltec.display -> display();
48
49     int n = WiFi.scanNetworks();
50     Heltec.display -> drawString(0, 25, "Completed Scanning");
51     Heltec.display -> display();
52     delay(500);
53     Heltec.display -> clear();
54
55     if (n == 0)
56     {
57         Heltec.display -> clear();
58     }
59 }

```

```

55 Heltec.display -> drawString(0, 0, "Unable to find connection");
56 Heltec.display -> display();
57 while(1);
58 }
59 else
60 {
61   Serial.print(n);
62   Heltec.display -> drawString(0, 0, (String)n);
63   Heltec.display -> drawString(14, 0, "networks found:");
64   Heltec.display -> display();
65   delay(500);
66
67   for (int i = 0; i < n; ++i) {
68     // Print the list of all networks available in the area
69     Heltec.display -> drawString(0, (i+1)*9, (String)(i + 1));
70     Heltec.display -> drawString(6, (i+1)*9, ":");
71     Heltec.display -> drawString(12, (i+1)*9, (String)(WiFi.SSID(i)));
72     Heltec.display -> drawString(90, (i+1)*9, " (");
73     Heltec.display -> drawString(98, (i+1)*9, (String)(WiFi.RSSI(i)));
74     Heltec.display -> drawString(114, (i+1)*9, ")");
75     // display.println((WiFi.encryptionType(i) ==
76     WIFI_AUTH_OPEN)? " WPA2" : " WPA");
77     delay(10);
78   }
79
80   Heltec.display -> display();
81   delay(800);
82   Heltec.display -> clear();
83 }
84
85 void setup()
86 {
87   Heltec.begin(true, false, true); // Enable Display, disable LoRa
88   // Enable, and enable Serial communication.
89   Heltec.display->clear();
90   WIFISetUp();
91
92   WiFi.disconnect(true); //WIFI
93   delay(1000);
94   WiFi.mode(WIFI_STA);
95   WiFi.setAutoConnect(true);
96 }
97 void loop()
98 {
99   WIFIScan();
100   delay(2000);
101 }

```

Listing 7: Code for testing the development board

In previous example, we tested the development board for its communication using WiFi and related functionalities. In this example, we test the LoRa communication functionality by sending data packets and receiving data packets over the LoRa network. To realize data transmission over LoRa, we use 865MHz channel which we can set using “LoRaWan Region ‘REGION_IN865’”

The listing for the same is given you in Listing.8. Line 6, initializes the

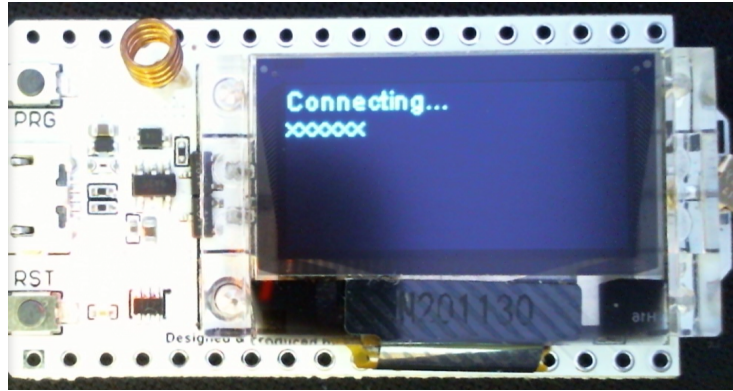


Figure 15: Performing Factory test on Heltec ESP32 LoRa Module

Display, LoRa, Serial-Communication, Antenna-Power-Boost, and selects the frequency BAND for the communication. User can defined display setting as per his/her choice in line 9 to 14. In `void loop()` function which runs repetitively has two parts; (a) status display of the packet transmission which is optional (b) data packet transmission at lines 30 to 34.

After uploading the firmware, the LCD screen on development board shows 'Heltec LoRa Initial Success!' indicating the LoRa functionalities are working without error as shown in the Fig.15. To test data transmission, we send some random data over the LoRa network as shown in Fig.16 and 17. Listing for testing LoRa by sending data packets:

```

1 #include "heltec.h"
2 #define BAND    865E6 //you can set band here directly ,e.g. 868E6,915
   E6
3 unsigned int counter = 0;
4 void setup()
5 {
6     Heltec.begin(true, true, true, true, BAND); // Initialize display,
       LoRa, serial comm., and power boost
7
8     // following block of a code is related to setting display
9     Heltec.display->init();
10    Heltec.display->flipScreenVertically();
11    Heltec.display->setFont(ArialMT_Plain_10);
12    Heltec.display->clear();
13    Heltec.display->drawString(0, 0, "Display Settings Completed!");
14    Heltec.display->display();
15    delay(2000);
16 }
17
18 void loop()
19 {
20     // following block of a code is related to displaying packet
       information
21     Heltec.display->clear();
22     Heltec.display->setTextAlignment(TEXT_ALIGN_LEFT);
23     Heltec.display->setFont(ArialMT_Plain_10);
24
25     Heltec.display->drawString(0, 0, "Sending packet: ");
26     Heltec.display->drawString(90, 0, String(counter));
27     Heltec.display->display();
28
29     // send packet
30     LoRa.beginPacket(); // begin data packet here
31     LoRa.setTxPower(14,RF_PACONFIG_PASELECT_PABOOST); // set transmit
       power
32     LoRa.print("hello "); // add data to be send into the packet
33     LoRa.print(counter); // add data to be send into the packet
34     LoRa.endPacket(); // end data packer here
35
36     counter++; // data for the LoRa packet.
37 }

```

Listing 8: Code for sending data packets over the LoRa Network

6 Display Programming

The visual displays are not necessary for sensor nodes but are very handy and helpful for troubleshooting and feedback. The Heltec ESP32 LoRa development board comes with inbuilt 0.96 Inch Blue OLED display and can be used in various development activities.

In this example, the rectangular graphics design is displayed to demonstrate the use of OLED display. Heltec Automations have developed various functions and display commands and are available at https://github.com/HelTecAutomation/Heltec_ESP32/blob/master/src/oled/OLEDDisplay.h. These

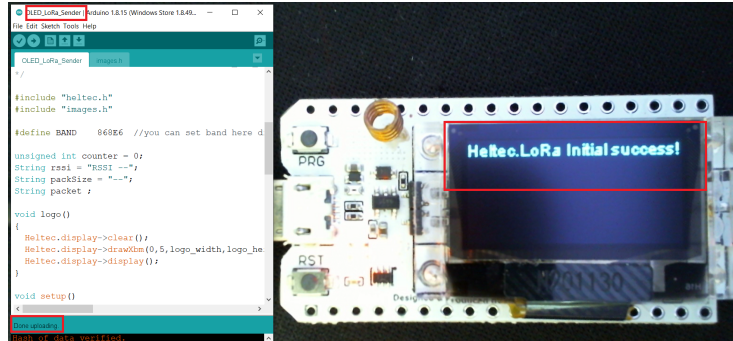


Figure 16: (a) Testing LoRa functionalities

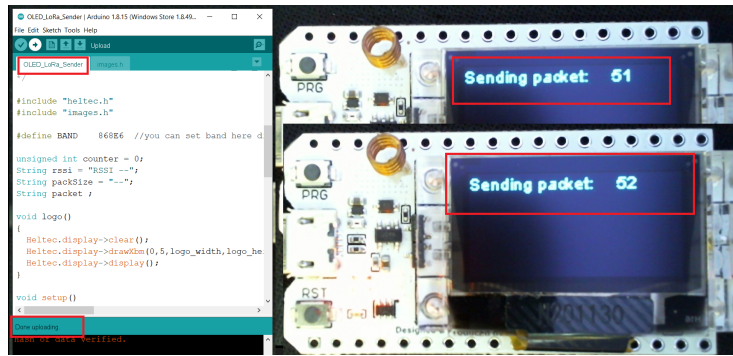


Figure 17: (b) Transmitting LoRa Packets

functions are compatible with arduino IDE and can be swiftly used for development. We describe the various graphics functions for drawing lines, rectangle, circle, filling the rectangle and custom function for printing the saved data into buffer.

- Listing for drawing rectangle on the OLED display.
We use `rawRect()` function to draw rectangle, the function definition for the same is given below; The function, uses two arguments; `DISPLAY_HEIGHT` and `DISPLAY_WIDTH` which are the height and width of the OLED display, instead user can use custom value for displaying the rectangle on the OLED display.

```

1      // setting done are considering the Adafruit_SSD1306 OLED
      display
2
3      void drawRect(void) {
4          for (int16_t i=0; i<DISPLAY_HEIGHT/2; i+=2) { //
      DISPLAY_HEIGHT and DISPLAY_WIDTH are the user arguments
5              Heltec.display->drawRect(i, i, DISPLAY_WIDTH-2*i,
      DISPLAY_HEIGHT-2*i); // drawRect function to draw rectangle.
6              Heltec.display->display();
7              delay(10);
8          }
9      }
10

```

Listing 9: Code for drawing rectangle on the OLED display

- Listing for filling rectangle

Same as the previous example, `fillRect()` accepts the arguments; `DISPLAY_HEIGHT` and `DISPLAY_WIDTH`, user can change the values according to the dimensions of the rectangle he/she wants to draw on the OLED display. The function definition for the same is given in Listing. 10. The color to fill can be selected using `setColour()` command as shown in line 12, here we select 'white' colour to fill the rectangle.

```

1      // setting done are considering the Adafruit_SSD1306 OLED
      display
2      void fillRect(void) {
3          uint8_t color = 1;
4          // DISPLAY_HEIGHT and DISPLAY_WIDTH are the user
      arguments
5          for (int16_t i=0; i<DISPLAY_HEIGHT/2; i+=3) {
6              Heltec.display->setColor((color % 2 == 0) ? BLACK :
      WHITE); // White color is used, alternate colors can be used
7              Heltec.display->fillRect(i, i, DISPLAY_WIDTH - i*2,
      DISPLAY_HEIGHT - i*2); // fillRect is the function to fill the
      rectangle
8              Heltec.display->display();
9              delay(10);
10             color++;
11         }
12         // Reset back to WHITE
13         Heltec.display->setColor(WHITE);
14     }
15

```

Listing 10: Code for filling rectangle on OLED display

- Listing for drawing circle on OLED display

To draw circles on the display we can use `drawCircle()` function as defined in Listing. 11. The function itself has two parts; (a) In first part, line 3 to 9 uses OLED dimensions and draw circle at an increment of every

2 pixels as given in the for loop at line 3. (b) The second part considers the circle as a combination of four segments drawn in four quadrants. The code in line 17 to 19 defines code for the segment in the first quadrant labeled as 0b00000001 and uses function `drawCircleQuads()`. The same code is repeated to draw rest of the segments to complete the circle as given in lines 20 to 27.

```

1      // setting done are considering the Adafruit_SSD1306 OLED
display
2      void drawCircle(void) {
3          for (int16_t i=0; i<DISPLAY_HEIGHT; i+=2) { \\ draw
circle for every 2 pixel increment
4              Heltec.display->drawCircle(DISPLAY_WIDTH/2,
DISPLAY_HEIGHT/2, i);\\ drawCircle is the fucnction to draw
circle
5              // DISPLAY_HEIGHT and DISPLAY_WIDTH are the user
arguments
6              Heltec.display->display();
7              delay(10);
8          }
9          delay(1000);
10         Heltec.display->clear();
11
12         // This will draw the part of the circle in quadrant 1
13         // Quadrants are numbered like this:
14         //   0010 | 0001
15         //   -----
16         //   0100 | 1000
17         // Every quadrant draw one segment of the circle
18         Heltec.display->drawCircleQuads(DISPLAY_WIDTH/2,
DISPLAY_HEIGHT/2, DISPLAY_HEIGHT/4, 0b00000001);
19         Heltec.display->display(); // draw one segment
20         delay(200);
21         Heltec.display->drawCircleQuads(DISPLAY_WIDTH/2,
DISPLAY_HEIGHT/2, DISPLAY_HEIGHT/4, 0b00000011);
22         Heltec.display->display(); // draw second segment
23         delay(200);
24         Heltec.display->drawCircleQuads(DISPLAY_WIDTH/2,
DISPLAY_HEIGHT/2, DISPLAY_HEIGHT/4, 0b00000111);
25         Heltec.display->display(); // draw third segment
26         delay(200);
27         Heltec.display->drawCircleQuads(DISPLAY_WIDTH/2,
DISPLAY_HEIGHT/2, DISPLAY_HEIGHT/4, 0b00001111);
28         Heltec.display->display(); // draw forth segment
29     }
30

```

Listing 11: Code for drawing circle on OLED display

- Listing for drawing lines on the OLED display.
Listing.12 presents the demo for graphics using lines drawn at different locations on the OLED display. `drawLines()` function executes this demo.

To draw a line on the OLED display, `drawLine()` function provided by heltec automation is used. The function `drawLine(0, 0, DISPLAY_WIDTH-1,i)` takes four arguments; first two arguments define the start point of a line and last two defines end point of a line. By varying starting and ending point of a line simple graphics is developed. In the Listing. 12, the function `drawLine()` is repeated multiple times as in line 4, 9, 17, 22, 30, 35, 42, and 47.

```

1      // Adapted from Adafruit_SSD1306
2      void drawLines() {
3          for (int16_t i=0; i<DISPLAY_WIDTH; i+=4) {
4              Heltec.display->drawLine(0, 0, i, DISPLAY_HEIGHT-1);
5              \\ drawLine is basic function for drawing line.
6              // takes four arguments; first two arguments define the
7              // start point of a line and last two defines end point of a
8              // line.
9              Heltec.display->display();
10             delay(10);
11         } // This loop display line starting from origin i.e.
12         // left hand top corner of OLED and rotate line along the width
13         // of OLED
14         for (int16_t i=0; i<DISPLAY_HEIGHT; i+=4) {
15             Heltec.display->drawLine(0, 0, DISPLAY_WIDTH-1, i);
16             Heltec.display->display();
17             delay(10);
18         } // This loop display line starting from origin i.e.
19         // left hand top corner of OLED and rotate line along the height
20         // of OLED
21         delay(250);
22
23         Heltec.display->clear();
24         for (int16_t i=0; i<DISPLAY_WIDTH; i+=4) {
25             Heltec.display->drawLine(0, DISPLAY_HEIGHT-1, i, 0);
26             Heltec.display->display();
27             delay(10);
28         } // This loop display line starting from coordinate (0,
29         // DISPLAY_HEIGHT-1) and rotate line along the width of OLED
30         for (int16_t i=DISPLAY_HEIGHT-1; i>=0; i-=4) {
31             Heltec.display->drawLine(0, DISPLAY_HEIGHT-1,
32             DISPLAY_WIDTH-1, i);
33             Heltec.display->display();
34             delay(10);
35         } // This loop display line starting from coordinate (0,
36         // DISPLAY_HEIGHT-1) and rotate line along the height of OLED
37         delay(250);
38
39         Heltec.display->clear();
40         for (int16_t i=DISPLAY_WIDTH-1; i>=0; i-=4) {
41             Heltec.display->drawLine(DISPLAY_WIDTH-1,
42             DISPLAY_HEIGHT-1, i, 0);
43             Heltec.display->display();
44             delay(10);
45         } // This loop display line starting from coordinate (
46         // DISPLAY_WIDTH-1, DISPLAY_HEIGHT-1) and rotate line along the
47         // width of OLED
48         for (int16_t i=DISPLAY_HEIGHT-1; i>=0; i-=4) {
49             Heltec.display->drawLine(DISPLAY_WIDTH-1,
50             DISPLAY_HEIGHT-1, 0, i);
51             Heltec.display->display();
52             delay(10);
53         } // This loop display line starting from coordinate (
54         // DISPLAY_WIDTH-1, DISPLAY_HEIGHT-1) and rotate line along the
55         // height of OLED
56         delay(250);
57         Heltec.display->clear();

```

```

42         for (int16_t i=0; i<DISPLAY_HEIGHT; i+=4) {
43             Heltec.display->drawLine(DISPLAY_WIDTH-1, 0, 0, i);
44             Heltec.display->display();
45             delay(10);
46         } // This loop display line starting from coordinate (
DISPLAY_WIDTH-1,0) and rotate line along the height of OLED
47         for (int16_t i=0; i<DISPLAY_WIDTH; i+=4) {
48             Heltec.display->drawLine(DISPLAY_WIDTH-1, 0, i,
DISPLAY_HEIGHT-1);
49             Heltec.display->display();
50             delay(10);
51         } // This loop display line starting from coordinate (
DISPLAY_WIDTH-1,0) and rotate line along the width of OLED
52         delay(250);
53     }
54

```

Listing 12: Code for drawing line on OLED

- Listing to develop custom function for storing and displaying the data on OLED display.

The buffer is used to store the data or characters to be displayed. To use this buffer, first we need to initialize it by defining its size in the form of number of lines and number of characters per line. For example in Listing. 13, 5 lines and 30 characters per line are declared as in line 4. Once, the size of buffer is declared, it can store the data inside it for printing and the same can be printed using command `drawLogBuffer()`.

```

1      void printBuffer(void) {
2          // Initialize the log buffer
3          // allocate memory to store 8 lines of text and 30 chars
4          // per line.
5          Heltec.display->setLogBuffer(5, 30);
6
7          // Some test data
8          const char* test[] = {
9              "Hello",
10             "World",
11             "____",
12             "Show off",
13             "how",
14             "the log buffer",
15             "is",
16             "working.",
17             "Even",
18             "scrolling is",
19             "working"
20         };
21
22         for (uint8_t i = 0; i < 11; i++) {
23             Heltec.display->clear();
24             // Print to the screen
25             Heltec.display->println(test[i]);
26             // Draw it to the internal screen buffer
27             Heltec.display->drawLogBuffer(0, 0);
28             // Display it on the screen
29             Heltec.display->display();
30             delay(500);
31         }
32     }

```

Listing 13: Code for develop custom function for storing and drawing the data on OLED display.

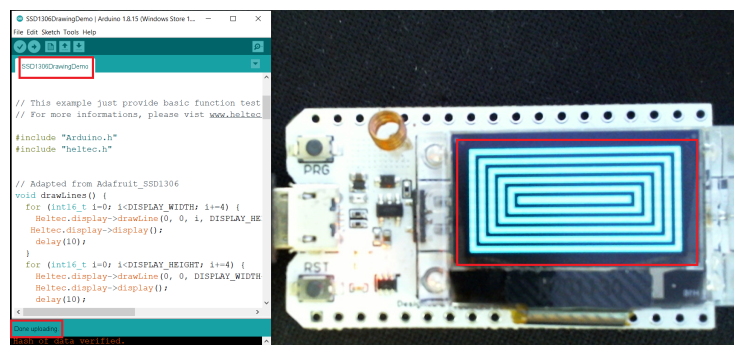


Figure 18: Displaying graphics on OLED display

7 Sensor Interfacing (data acquisition)

Data acquisition from the field is one of the important rather primary requirement of any IoT system. This is achieved by deploying sensor nodes at required points in the field. The sensor node is a hardware with sensors, a processor, battery and communication technology either wifi, bluetooth, LoRa, or RF. Following are the few characteristics of a typical sensor node in a IoT framework.

- It should support Analog and Digital Sensors.
- Processor should support a low or ultra low power configuration.
- Processor should be powerful enough to handle preprocessing on the sensor data if required.
- Communication technology should be power efficient.
- Sensor node should run sufficiently long enough on a single charge of a battery.

In this section, a sensor node using ESP32 LoRa development board is demonstrated as it support both analog and digital sensors, have ESP32 with ultra low power core and enabled with long range RF (LoRa) communication support and thus become eligible option for using it as a sensor node in IoT framework.

We interface BMP280 sensor [4] from Bosch with Heltec development board to measure Temperature and Pressure. We display these readings on OLED display and also transmit over the LoRa network.

The listing performs in three stages; reading the sensor value, displaying it on the OLED display and lastly, transmitting these values over the LoRa Network.

The function, `drawFrontFaceDemo()` accept four arguments Temperature, Pressure, Altitude, and Real Altitude and display these values on the OLED display using function `drawString(arg1, arg2, arg3)` in which, `arg1` is starting column, `arg2` is starting line and `arg3` is the text itself.

In `void loop()`, at line 40, we check the sensor availability using `!bmp.begin()` and if the sensor is available then we read the values of Temperature, Pressure, Altitude, and Real Altitude at lines, 44, 49, 54, and 65 respectively. The function to read the sensor values is provided by the library `BMP180.h` which we included at the top of the listing.

Lastly, we want to send these sensor values over the LoRa Network. We create a data packet for transmission at lines 75 to 82. `LoRa.beginPacket()` initiates the data packet and then `LoRa.print()` command adds sensor data into the packet. Finally, `LoRa.endPacket()` ends the packet and transmit it on the LoRa network.

Listing for testing LoRa by sending data packets:

```

1 #include "Arduino.h"
2 #include "heltec.h"
3 #include <Wire.h>
4 #include <BMP180.h>
5 #include "string.h"
6
7 #define BAND 865E6 // Set LoRa band (here Indian)
8
9 BMP085 bmp;
10
11 uint8_t T[20] = {"Temperature"};
12
13 void setup() {
14   pinMode(Vext,OUTPUT);
15   digitalWrite(Vext,HIGH);
16   Serial.begin(115200); // initialize serial comm. at a baud rate of
17                           115200
18 }
19 void drawFontFaceDemo(double T,double P,double A,double R) {
20   //this block of code print the sensor values on the OLED display
21   Heltec.begin(true, false, true);
22   // Heltec.display->flipScreenVertically();
23   Heltec.display->clear();
24   Heltec.display->setTextAlignment(TEXT_ALIGN_LEFT);
25   Heltec.display->setFont(ArialMT_Plain_10);
26   Heltec.display->drawString(0, 0, "Temperature =");
27   Heltec.display->drawString(76, 0, (String)T);
28   Heltec.display->drawString(106, 0, " *C");
29   Heltec.display->drawString(0, 16, "Pressure =");
30   Heltec.display->drawString(56, 16, (String)P);
31   Heltec.display->drawString(100, 16, "Pa");
32   Heltec.display->drawString(0, 32, "Altitude =");
33   Heltec.display->drawString(56, 32, (String)A);
34   Heltec.display->drawString(86, 32, " m");
35   Heltec.display->drawString(0, 48, "Real altitude =");
36   Heltec.display->drawString(76, 48, (String)R);
37   Heltec.display->drawString(106, 48, " m");
38   Heltec.display->display();
39 }
40 void loop() {
41   if (!bmp.begin()) { // check the sensor availability
42     Serial.println("Could not find a valid BMP085 sensor, check wiring!");
43     ;
44   while (1) {}
45   }
46   double T =bmp.readTemperature(); // read Temperature value from
47   sensor
48   Serial.print("Temperature = ");
49   Serial.print(T);
50   Serial.println(" *C");
51
52   double P = bmp.readPressure(); // read Pressure value from sensor
53   Serial.print("Pressure = ");
54   Serial.print(bmp.readPressure());
55   Serial.println(" Pa");

```

```

55     double A = bmp.readAltitude(); // read Altitude value from sensor
56     // Standard Altitude calculation
57     Serial.print("Altitude = ");
58     Serial.print(bmp.readAltitude());
59     Serial.println(" meters");
60
61
62     Serial.print("Sea-level Pressure (calculated) = ");
63     Serial.print(bmp.readSealevelPressure()); // read sea level
64     // pressure from the sensor and display on serial terminal.
65     Serial.println(" Pa");
66
67     double R = bmp.readAltitude(101500); // read altitude at 101500 sea
68     // level pressure value from sensor
69     // Precise altitude calculation
70     Serial.print("Real altitude = ");
71     Serial.print(bmp.readAltitude(101500));
72     Serial.println(" meters");
73
74     Serial.println();
75     delay(500);
76     drawFontFaceDemo(T, P, A, R) ; // call function to display data on
77     // OLED
78     // send packet
79     LoRa.beginPacket(); // starts packet
80
81     LoRa.setTxPower(14,RF_PACONFIG_PASELECT_PABOOST);
82     LoRa.print(T); // add sensor value to data packet
83     LoRa.print(P);
84     LoRa.print(A);
85     LoRa.print(R);
86     LoRa.endPacket(); // ends packet and transmit
87     delay(5000);
88 }

```

Listing 14: Code for displaying and trans

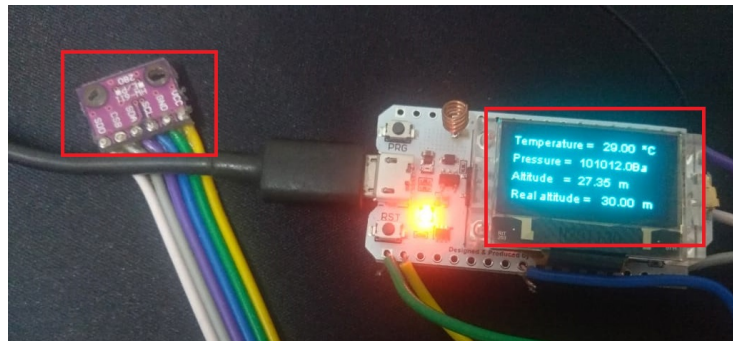


Figure 19: Displaying readings from BMP280

8 Applications in Home Automation, Industry and Smart Agriculture

The reliable and long distance connectivity is of highest concern in IoT networks, especially for industry 4.0 and smart agriculture long range wireless IoT connectivity is of prime interest due to large deployment area. LoRa and LoRa Wide Area Network (LoRaWAN) are ideal for such applications. They cover a long range at a expense of little power and low bandwidth such networks are commonly known as Low-Power, Wide-Area Networks (LPWAN). LoRaWAN stands a as number one in the non-cellular LPWA technologies and thus are a best fit for smart cities, environmental monitoring, smart agriculture and far more, including industry 4.0 [8] as illustrated in Fig.20.

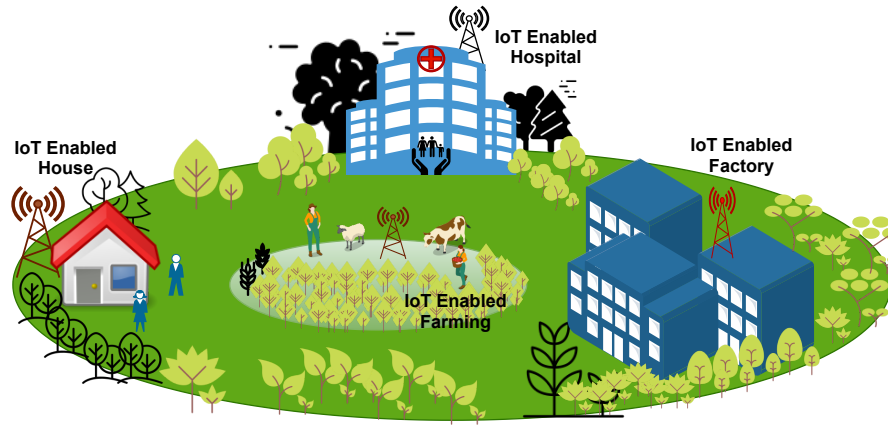


Figure 20: IoT applications and ecosystem

8.0.1 Real life LoRa Application for Smart Agriculture

In the previous sections, we have seen all the elements for development of LoRa based IoT application and associated coding. This section presents end-to-end application for monitoring Temperature, Humidity and Pressure in the agriculture field. We deploy BME280 sensor [1] interfaced with Heltec ESP32 LoRa development board in the field to capture Temperature, Humidity and Pressure. The readings from the BME280 are transmitted over the 865MHz LoRa channel. The receiver is the another Heltec ESP32 LoRa development board programmed to receive the readings and display on OLED display. Below is the complete code for transmitter and receiver.

Code for Weather Station using ESP32 LoRa Heltec Development Board

Listing for transmitter:

To interface the BME280 sensor with heltec LoRa module, we include header files `wire.h`, `Adafruit_Sensor.h`, and `Adafruit_BME280.h`. In `void setup()` function, we enable OLED Display, LoRa, Serial-Communication, Antenna-Power-Boost, and selects the frequency BAND for the communication using `Heltec.begin()` and then we define the settings for OLED display at lines 21 to 25. The function `drawFrontFaceDemo()` is defined to display sensor values on OLED display and then we read the sensor values and transmit them over the LoRa network as explained in the previous example.

```

1
2 #include "Arduino.h"
3 #include "heltec.h"
4 // header files corresponding to libraries of the sensor used
5 #include <Wire.h>
6 #include <Adafruit_Sensor.h>
7 #include <Adafruit_BME280.h>
8 #include "string.h"
9
10 #define SEALEVELPRESSURE_HPA (1013.25)
11 Adafruit_BME280 bme;
12
13
14 #define BAND      865E6 // Set LoRa band (here Indian)
15
16 //uint8_t T[20] = {"Temperature"};
17
18 void setup() {
19   pinMode(Vext,OUTPUT);
20   digitalWrite(Vext,LOW);
21   // following block of code heltec module and display
22   Heltec.begin(true, true, true, true, BAND);
23   Heltec.display->init();
24   Heltec.display->flipScreenVertically();
25   Heltec.display->setFont(ArialMT_Plain_10);
26   Heltec.display->drawString(0, 0, "LoRa Initial success!");
27
28   Serial.begin(115200); // begin serial comm.
29
30 }
31 // function to display sensor values on the OLED display
32 void drawFontFaceDemo(double T,double P,double H) {
33
34   Heltec.display->drawString(0 , 0 , "Temperature =");
35   Heltec.display->drawString(76 , 0 , (String)T);
36   Heltec.display->drawString(106 , 0 , " *C");
37   Heltec.display->drawString(0 , 16 , "Pressure =");
38   Heltec.display->drawString(56 , 16 , (String)P);
39   Heltec.display->drawString(100 , 16 , "Pa");
40   Heltec.display->drawString(0 , 32 , "Humidity =");
41   Heltec.display->drawString(56 , 32 , (String)H);
42   Heltec.display->drawString(86 , 32 , "%");
43   Heltec.display->display();
44 }
45
46 void loop() {
47   if (!bme.begin()) { // check sensor availability
48     Serial.println("Could not find a valid BME280 sensor, check wiring!");
49     ;
50     while (1) {}
51   }
52   // read the sensor values
53   double T =bme.readTemperature();
54   double P = bme.readPressure();
55   double H = bme.readHumidity();
56
57   drawFontFaceDemo(T, P, H) ; // call function to display data on OLED

```

```

57 // send packet
58 LoRa.beginPacket(); // beigns the data packet
59 LoRa.setTxPower(14,RF_PACONFIG_PASELECT_PABOOST);
60 LoRa.print("Weather Station");
61 LoRa.print(T); // add data values into data packet
62 LoRa.print(P);
63 LoRa.print(H);
64 LoRa.endPacket(); // ends the data packet and trasmit
65
66 delay(5000);
67
68 }

```

Listing 15: Code for transmitter using MBE280 and ESP32 LoRa Heltec Development Board

Listing for receiver:

The listing for receiver involves setting an OLED display to show received sensor reading in function `LoRaData()` and parse the data packet to extract packet size and data itself. In `void setup()`, we receive the LoRa data packet at line 29 using `LoRa.receive()`. At lines 33 and 34, we extract the packet size and call function `void cbk()` defined at line 13 which extracts the data and save it into a variable 'packet' at line 15 and display it by calling function `LoRaData()`.

```

1 #include "heltec.h"
2 #define BAND 865E6 // Set LoRa band (here Indian)
3 String packet ;
4
5 void LoRaData(){ // setup of OLED
6   Heltec.display->clear();
7   Heltec.display->setTextAlignment(TEXT_ALIGN_LEFT);
8   Heltec.display->setFont(ArialMT_Plain_10);
9   Heltec.display->drawStringMaxWidth(0 , 26 , 128, packet);
10  Heltec.display->display();
11 }
12
13 void cbk(int packetSize) { // extract the data from the complete packet
14   packet = ""; // hold the data
15   for (int i = 0; i < packetSize; i++) { packet += (char) LoRa.read();
16   }
17   LoRaData(); // print the data on the OLED
18 }
19
20 void setup() {
21   Heltec.begin(true, true, true, true, BAND); // begin heltec module
22
23   Heltec.display->init();
24   Heltec.display->flipScreenVertically();
25   Heltec.display->setFont(ArialMT_Plain_10);
26   Heltec.display->drawString(0, 0, "LoRa has been initialized.");
27   Heltec.display->drawString(0, 10, "Wait for incoming data...");
28   Heltec.display->display();
29   delay(1000);
30   LoRa.receive(); // receive the data from LoRa network
31 }
32
33 void loop() {
34   int packetSize = LoRa.parsePacket(); // parse the received packet
35   if (packetSize) { cbk(packetSize); } // calculate packet size
36   delay(10);
37 }

```

Listing 16: Code for Receiver using ESP32 LoRa Heltec Development Board

9 Conclusions

LoRa is a promising and futuristic technology for IoT based applications with a wide range of real-world applications. This chapter provides a practical getting-started resource for LoRa communication. We begin by the need and salient features of LoRa followed by an overview of Heltec LoRa test platform. We explored the basic functionalities of the platform, a simple transmitter-receiver communication model and data acquisition modules. Further, the applications related to home automation and IoT smart agriculture are presented. LoRa can provide a good coverage of signals and are suitable for low-throughput, low-power, and long-range networks.

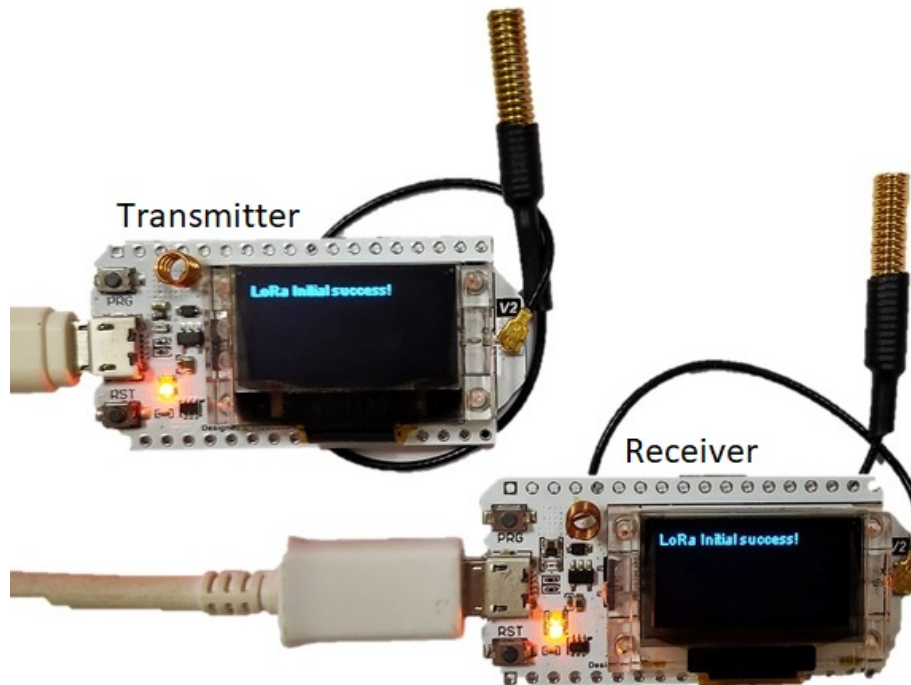


Figure 21: A simple LoRa transmitter-receiver model

References

- [1] Adafruit. Bme280 datasheet. <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-bme280-humidity-barometric-pressure-temperature-sensor-breakout.pdf>, 2021. Accessed: 31-08-2021.
- [2] LoRa Alliance. Rp002-1.0.2 lorawan® regional parameters. https://loro-alliance.org/wp-content/uploads/2020/11/RP_2-1.0.2.pdf, October 2020. Accessed: 29-08-2021.
- [3] Arduino. Guide. <https://www.arduino.cc/>, February 2018. Accessed: 30-08-2021.
- [4] BOSCH. Bmp280 datasheet. <https://cdn-shop.adafruit.com/datasheets/BST-BMP280-DS001-11.pdf>, May 2015. Accessed: 31-08-2021.
- [5] ESPRESSIF. *ESP32 Series Datasheet*, 4 2021. Rev. 3.7.
- [6] Heltec. Wifi lora 32(v2.1) pinout diagram. https://resource.heltec.cn/download/WiFi_LoRa_32/WIFI_LoRa_32_V2.pdf, January 2020. Accessed: 29-08-2021.

- [7] HelTecAutomations. Heltec docs. https://github.com/HelTecAutomation/HeltecDocs/blob/master/en/source/esp32/quick_start.md, May 2019. Accessed: 30-08-2021.
- [8] iSCOOP. Lora and lorawan: the technologies, ecosystems, use cases and market. <https://www.i-scoop.eu/internet-of-things-iot/lpwan/iot-network-lora-lorawan/>, February 2020. Accessed: 30-08-2021.
- [9] Semtech. Lora® and lorawan®:a technical overview. Technical report, Semtech Corporation, 2020. Accessed: 29-08-2021.